



Módulo 3 · Testing, Debugging y Documentación con IA

Pipelines de Testing Automatizado — Fundamentos e IA

Ejercicio de Clase · Ejercicio 1 de 2 · Bloques 4.1 — 4.2 — 4.3

Nombre del alumno/a	ANGEL ECHENIQUE
Fecha	25/03/2026
 Enlace GitHub	

 **Instrucciones:** Responde con lo que hemos visto en clase. Puedes consultar tus apuntes. En el Bloque 5 deberás investigar por tu cuenta y citar la fuente. ¡Solo demostrar que entendiste!

BLOQUE 1 — ¿Qué es un pipeline de testing automatizado? (4.1)

En clase vimos que un pipeline es un flujo de trabajo estructurado con varias etapas secuenciales. Completa la siguiente tabla con la información de cada etapa:

Nº	Nombre de la etapa	¿Qué hace exactamente?	Herramientas o comandos
0	Disparador	Inicia el pipeline cuando haces push o commit	git push, webhook
1	Instalación de dependencias	Instala librerías necesarias del proyecto	pip install -r requirements.txt, npm install
2	Análisis estático	Revisa el código sin ejecutarlo (errores, estilo, malas prácticas)	flake8, eslint
3	Pruebas unitarias	Comprueba funciones individuales	pytest, unittest
4	Pruebas de integración	Verifica que los módulos funcionan juntos	pytest, supertest
5	Análisis de seguridad	Detecta vulnerabilidades	bandit, snyk
6	Construcción (Build)	Genera el paquete o app lista para usar	docker build, npm build

7	Despliegue (Deploy)	Sube la aplicación a producción o servidor	docker push, deploy script
---	---------------------	--	----------------------------

1.2 CI/CD: Concepto integrado

Explica con tus propias palabras la diferencia entre CI y CD. Incluye un ejemplo práctico de cada uno:

Concepto	Explicación con tus palabras + ejemplo
CI — Integración Continua	Es el proceso de integrar cambios de código frecuentemente y ejecutar pruebas automáticamente. Ejemplo: al hacer push en GitHub, se ejecutan tests con GitHub Actions.
CD — Despliegue Continuo	Es el proceso de desplegar automáticamente el código una vez pasa todas las pruebas. Ejemplo: si todos los tests pasan, la aplicación se despliega automáticamente en un servidor.

1.3 — Idea clave: ¿Qué pasa si una etapa falla?

En el pipeline, cada etapa actúa como un filtro de calidad. Describe qué ocurre cuando falla cada una de estas etapas y qué consecuencia tiene para el equipo:

Etapas que falla	Consecuencia y qué debe hacer el equipo
Tests unitarios fallan	Significa que hay errores en funciones individuales, el equipo debe corregir la lógica antes de continuar.
Análisis de seguridad detecta riesgo	Puede haber vulnerabilidades graves, el equipo debe corregirlas antes de desplegar para evitar ataques.
Construcción (build) falla	El proyecto no puede generarse correctamente, el equipo debe revisar dependencias o errores de configuración.

BLOQUE 2 — ¿Por qué integrar IA en un pipeline? (4.2)

En clase vimos que la IA puede transformar un pipeline tradicional en un sistema inteligente. Completa la tabla con las funciones clave de la IA y para qué sirve cada una:

Función clave de la IA	¿Para qué sirve?	¿Qué problema resuelve?
Generación automática de tests	Crea pruebas sin escribirlas manualmente.	Ahorra tiempo y evita olvidar casos importantes.
Explicación de fallos del pipeline	Traduce errores técnicos a lenguaje entendible.	Evita perder horas interpretando logs.
Detección de vulnerabilidades	Analiza el código en busca de fallos de seguridad.	Previene ataques o fallos en producción.
Optimización de rendimiento	Sugiere mejoras en código y ejecución.	Reduce consumo de recursos y mejora velocidad.
Análisis de logs complejos	Interpreta grandes cantidades de logs automáticamente.	Evita análisis manual largo y confuso.
Propuestas de refactorización	Sugiere mejoras en estructura del código.	Mejora mantenibilidad y calidad del software.
Reportes automáticos	Genera informes claros del estado del pipeline.	Facilita la comunicación en el equipo.

2.2 Pipeline tradicional vs. Pipeline con IA

Vimos una tabla comparativa en clase. Completa la columna 'Con IA' para cada situación:

Situación	Sin IA	Con IA
Test que falla	Solo muestra el mensaje de error en la consola.	Explica la causa del error y sugiere cómo solucionarlo.
Código nuevo subido	Ejecuta los tests existentes.	Genera nuevos tests automáticamente y detecta casos no cubiertos.
Resultado del pipeline	"Todos los tests pasan".	Genera un informe detallado con métricas, cobertura y posibles mejoras.

Debug de un error	El equipo revisa los logs manualmente durante horas.	La IA analiza logs y señala directamente el origen del problema.
Seguridad	Escaneo básico de vulnerabilidades conocidas	Detecta vulnerabilidades avanzadas y propone soluciones.

2.3 — El ahorro de tiempo

Según lo visto en clase, la IA puede reducir hasta el 78% el tiempo dedicado a resolver un fallo. Escribe en qué fases del proceso se ahorra más tiempo y por qué:

Leer logs minimo 1 hora revisando, buscando errores o el error se puede tardar mas o menos 2 horas con suerte y luego nos tocaria probar las soluciones que tengamos para ese problema en cuestion a esto le dedicaremos 2 a 3 horas como minimo. Para un total de 5 o 6 horas.
En estas fases es, en la que nos ahorraria mas tiempo la IA. Porque nos diria en minutos el problema = donde esta - como solucionarlo y ayuda a la calidad del software.
Esto reduce significativamente el tiempo de desarrollo y resolución de fallos.

BLOQUE 3 — Herramientas CI/CD que soportan IA (4.3)

En clase repasamos las principales plataformas CI/CD y su compatibilidad con la IA. Completa la tabla:

Plataforma	¿Qué capacidades IA tiene?	Nivel de integración (Alto/Medio)	Observación clave
GitHub Actions	Integración con IA para generar tests, revisar código y automatizar workflows.	Alto	Muy integrado con herramientas como Copilot.
GitLab CI	IA para análisis de seguridad, testing y DevSecOps.	Alto	Fuerte enfoque en seguridad automatizada.
Azure DevOps	Integración con servicios de IA de Microsoft.	Alto	Potente en entornos empresariales.
Jenkins	IA mediante plugins externos.	Medio	No tiene IA nativa, requiere configuración.

CircleCI	Integración con herramientas externas de IA.	Medio	Flexible pero menos integrado.
----------	--	-------	--------------------------------

3.2 Buenas prácticas al elegir herramientas CI/CD con IA

Vimos 7 buenas prácticas en clase. Escríbelas y explica brevemente para qué sirve cada una:

Nº	Buena práctica	¿Para qué sirve?
1	Revisar resultados de la IA	Evita errores generados automáticamente.
2	No depender totalmente de la IA	Mantiene el control humano del sistema.
3	Integrar IA progresivamente	Reduce riesgos en el pipeline.
4	Usar herramientas compatibles	Asegura mejor integración.
5	Monitorizar resultados	Detecta fallos o comportamientos incorrectos.
6	Mantener seguridad	Evita vulnerabilidades introducidas por automatización.
7	Documentar el uso de IA	Facilita mantenimiento y comprensión del sistema.

3.3 — Verdadero o Falso

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación	Justificación (solo si es Falsa)
V	GitHub Actions es compatible con GitHub Copilot para generar y revisar tests.	
F	Jenkins integra IA de forma nativa sin necesidad de configuración adicional.	Jenkins necesita plugins externos para usar IA.
V	GitLab CI tiene IA integrada en su modelo DevSecOps para escaneo automático.	

F	CircleCI y Jenkins tienen nivel de integración con IA equivalente a GitHub Actions.	Tienen menor nivel de integración y dependen de herramientas externas.
V	Azure DevOps puede integrarse con modelos de IA de Microsoft.	

BLOQUE 4 — La analogía del inspector de fábrica — Síntesis

📌 Analogía de clase: El pipeline es una fábrica donde cada etapa revisa la pieza. La IA actúa como un inspector experto que analiza, predice, corrige y mejora continuamente.

Relaciona cada característica del inspector con su equivalente en la IA integrada en el pipeline:

Característica del inspector de fábrica	¿Cómo lo hace la IA en el pipeline?
Examina cada pieza al detalle	Analiza el código línea por línea detectando errores, malas prácticas y posibles fallos
Detecta defectos que las máquinas no ven	Identifica bugs complejos, casos edge y problemas lógicos que los tests tradicionales no detectan
Propone mejoras en el diseño	Sugiere refactorización, optimización de código y mejores prácticas
Anticipa fallos antes de que ocurran	Predice errores basándose en patrones y analiza riesgos antes de ejecutar el código
Aprende con cada revisión y se vuelve más preciso	Mejora sus recomendaciones usando datos históricos y patrones de errores

4.2 — Pregunta de síntesis

¿Por qué el pipeline tradicional se describe como 'reactivo' y el pipeline con IA como 'proactivo'?
Explicalo con tus propias palabras usando ejemplos concretos de lo que vimos en clase:

El reactivo: Actúa cuando ya hay un error, por ejemplo: Si el test falla, lo sabremos después. Entonces aquí tenemos que investigarlo manualmente que ha pasado.
y

El proactivo: Reacciona al error y se anticipa al problem, nos dice esto va a fallar antes de ejecutarlo. Por ejemplo, puede analizar el código antes de ejecutar los tests, detectar posibles errores o vulnerabilidades y sugerir soluciones antes de que el fallo ocurra.

BLOQUE 5 — Investigación propia (cita la fuente)

Investiga: Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable (documentación oficial, artículo técnico, blog de empresa tecnológica...) y cita siempre de dónde lo sacaste.

5.1 — Herramienta real de IA en pipelines

Investiga una herramienta real que integre IA en pipelines CI/CD que NO hayamos visto en clase (ejemplos: Snyk, Mend, DeepCode, Codacy, SonarCloud con IA...). Describe sus funcionalidades principales y cómo se integra:

SonarCloud es una herramienta de análisis de código que utiliza inteligencia artificial y análisis avanzado para detectar errores, vulnerabilidades y problemas de calidad en el código de forma automática dentro de pipelines CI/CD.

Permite:

- Detectar bugs y vulnerabilidades
- Analizar cobertura de tests
- Sugerir mejoras de código
- Integrarse automáticamente en pipelines

Campo	Tu respuesta
Nombre de la herramienta	SonarCloud
URL o fuente consultada	https://sonarcloud.io
Fecha de consulta	25/03/2026
¿Con qué plataforma CI/CD es compatible?	GitHub Actions, GitLab CI, Azure DevOps, Bitbucket

5.2 — Un caso real de empresa

Busca un caso real de una empresa que haya integrado IA en su pipeline de CI/CD. Describe brevemente qué problema tenían, qué solución implementaron y qué resultados obtuvieron:

Netflix implementó inteligencia artificial en sus pipelines de CI/CD para mejorar la detección de fallos y optimizar el rendimiento de sus sistemas.

Problema:

Tenían sistemas muy complejos y necesitaban detectar errores rápidamente antes de afectar a millones de usuarios.

Solución:

Integraron herramientas automatizadas con capacidades de análisis inteligente para monitorizar logs, detectar anomalías y anticipar fallos.

Resultados:

- Reducción del tiempo de detección de errores
- Mayor estabilidad en producción
- Mejora en la calidad del software

Campo	Tu respuesta
Empresa o proyecto	Netflix
Fuente	https://netflixtechblog.com
Fecha de consulta	25/03/2026

